ASSIGNMENT 21.2 AI-ASSISTED-CODING 2303A52490

Task 1: AI-Assisted Pair Programming

Initial program generated with AI help - simple To-Do List Manager.

```
# todo.py - Initial version from AI
todos = []

def add_task(task):
    todos.append(task)
    print(f"Added: {task}")

def view_tasks():
    for i, task in enumerate(todos, 1):
        print(f"{i}. {task}")

def remove_task(index):
    if 0 < index <= len(todos):
        removed = todos.pop(index - 1)
        print(f"Removed: {removed}")
    else:
        print("Invalid index")

while True:
    print("\n1. Add 2. View 3. Remove 4. Quit")
    choice = input("Choose: ")
    if choice == '1':
        task = input("Task: ")
        add_task(task)
    elif choice == '2':
        view_tasks()
    elif choice == '3':
        view_tasks()
        idx = int(input("Index to remove: "))
        remove_task(idx)
    elif choice == '4':
        break
```

AI suggested enhancements: add error handling, save/load from file, mark complete.

Improved version after AI suggestions:

```python
# todo_improved.py - Enhanced with AI help: error handling, persistence, complete feature
import json
import os

TODO_FILE = 'todos.json'

def load_todos():
    if os.path.exists(TODO_FILE):
        with open(TODO_FILE, 'r') as f:
            return json.load(f)
    return []

def save_todos(todos):
    with open(TODO_FILE, 'w') as f:
        json.dump(todos, f)

todos = load_todos()

def add_task(task):
    todos.append({"task": task, "completed": False})
    save_todos(todos)
    print(f"Added: {task}")

def view_tasks():
    if not todos:
        print("No tasks.")
        return
    for i, item in enumerate(todos, 1):
        status = "✓" if item["completed"] else " "
        print(f"{i}. [{status}] {item['task']}")

def remove_task(index):
    try:
        if 0 < index <= len(todos):
            removed = todos.pop(index - 1)
            save_todos(todos)
            print(f"Removed: {removed['task']}")
        else:
            print("Invalid index")
    except ValueError:
        print("Enter valid number")

def complete_task(index):
    try:
```

```
        if 0 < index <= len(todos):
            todos[index - 1]["completed"] = True
            save_todos(todos)
            print("Marked complete")
        else:
            print("Invalid index")
    except ValueError:
        print("Enter valid number")

while True:
    print("\n1. Add 2. View 3. Remove 4. Complete 5. Quit")
    choice = input("Choose: ")
    if choice == '1':
        task = input("Task: ").strip()
        if task:
            add_task(task)
        else:
            print("Task cannot be empty")
    elif choice == '2':
        view_tasks()
    elif choice == '3':
        view_tasks()
        idx = int(input("Index: "))
        remove_task(idx)
    elif choice == '4':
        view_tasks()
        idx = int(input("Index: "))
        complete_task(idx)
    elif choice == '5':
        break
    else:
        print("Invalid choice")
```

Comparison: Original was basic console app without persistence or completion. AI added JSON save/load, completion status, input validation - makes it production-ready.

Task 2: Version Control Integration

Initialized Git repo for the project.

```
$ mkdir todo-ai-lab
$ cd todo-ai-lab
$ git init
```

```
Initialized empty Git repository in /home/user/todo-ai-lab/.git/

$ git branch -M main
$ git add todo.py
$ git commit -m "Initial AI-generated To-Do List Manager"
[main (root-commit) abc1234] Initial AI-generated To-Do List Manager
 1 file changed, 25 insertions(+)
 create mode 100644 todo.py
```

Created new branch for improvements.

```
$ git checkout -b ai-enhancements
Switched to a new branch 'ai-enhancements'

# Copied improved code to todo.py, added improvements
$ git add todo.py
$ git commit -m "AI-assisted enhancements: persistence, completion, error handling"
[ai-enhancements def5678] AI-assisted enhancements: persistence, completion, error handling
 1 file changed, 30 insertions(+), 10 deletions(-)
```

Merged back to main.

```
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'main'.

$ git merge ai-enhancements
Updating abc1234..def5678
Fast-forward
 todo.py | 40 ++++++++++++++++++++----------------------
 1 file changed, 20 insertions(+), 20 deletions(-)
```

Reflection: Git branching allowed safe testing of AI changes without breaking main. Merging integrated improvements cleanly, simulating pair programming collab.

Task 3: AI for Commit Message Generation

Changes made: Added persistence with JSON.

Human-written commit: "add file save"

Another example:

Changes: Added completion toggle.

Human: "add done feature"

AI: "feat(todo): introduce completion status with toggle function and visual indicator in list view"

AI messages are more descriptive, use conventional commits (feat:, fix:), improve traceability in git log.

```
$ git log --oneline
def5678 (HEAD -> main) feat(persistence): add JSON file saving/loading...
abc1234 Initial AI-generated To-Do List Manager
```

Better docs for team reviews.

Task 4: Pair Programming Simulation

Student A initial code (simple calculator instead of todo for variety):

```
# calc_initial.py by Student A
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

print(add(5, 3))  # 8
print(sub(5, 3))  # 2
```

Student B with AI: Suggested menu, error handling, mul/div.

Improved:

```
# calc_improved.py
def add(a, b): return a + b
def sub(a, b): return a - b
def mul(a, b): return a * b
def div(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: Division by zero"
```

```
while True:
    print("1.Add 2.Sub 3.Mul 4.Div 5.Quit")
    ch = input()
    if ch == '5': break
    a, b = map(float, input("Nums: ").split())
    if ch == '1': print(add(a,b))
    elif ch == '2': print(sub(a,b))
    elif ch == '3': print(mul(a,b))
    elif ch == '4': print(div(a,b))
```

Git workflow:

```
$ git init calc-lab
$ git add calc_initial.py
$ git commit -m "Student A: basic add/sub calc"

$ git checkout -b student-b-ai
$ mv calc_improved.py calc.py  # rename and replace
$ git add calc.py
$ git rm calc_initial.py
$ git commit -m "Student B: AI-enhanced calculator with menu, mul/div, errors"
$ git checkout main
$ git merge student-b-ai
Auto-merging calc.py
```

Collaboration: Student A provided base, B+AI polished it. Git tracked contributions clearly.